Porting Informath to a New Language

GF-RGL and Claude: a Finnish case study

Aarne Ranta

July 2026

Outline

What Informath is

The problem

Mathematics exists in two registers.

Formal

- Agda, Rocq, Lean, Dedukti
- machine-checkable
- unreadable to most humans

Informal

- English, French, German, Swedish
- readable
- ambiguous, unverifiable

Informath translates between them — in *all* directions.

Four directions

- formal → informal (**informalization**)
- informal → formal (**autoformalization**)
- informal → informal (translation, *via* the formal)
- formal → formal (in special cases)

Two goals:

1. complete informalization of anything expressible in Dedukti;
2. generate *and analyse* informal mathematical language, in all of its variation.

Inspired by Ganesalingam, *The Language of Mathematics* (2013).

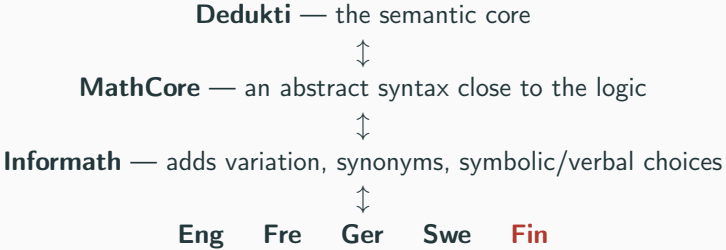
One statement, many languages

```
Dedukti: prop110 : (a : Elem Int) -> (c : Elem Int) ->
  Proof (and (odd a) (odd c)) ->
  Proof (forall Int (b => even (plus (times a b) (times b c)))).
```

- **En**: Let a and c be integers. Assume that a and c are odd. Then $ab + bc$ is even for all integers b.
- **De**: Seien a und c ganze Zahlen. Nimm an, dass a und c ungerade sind. Dann ist $ab + bc$ gerade für jede ganze Zahl b.
- **Sv**: Låt a och c vara heltal. Anta att a och c är udda. Då är $ab + bc$ jämnt för alla heltal b.
- **Fi**: — *what this talk adds*

All generated from the Dedukti source. Agda, Rocq and Lean too.

Architecture



Agda, Rocq and Lean hang off Dedukti by (partial) conversions.

The natural languages hang off **Informath** by GF concrete syntaxes.

GF in five minutes

Abstract vs. concrete syntax

GF separates *what can be said* from *how it is said*.

```
abstract MathCore = {
  cat Prop ; Kind ; Ident ;
  fun CoreAllProp : Kind -> Ident -> Prop -> Prop ;
}
```

```
concrete MathCoreEng of MathCore = {
  lincat Prop = S ;
  lin CoreAllProp kind x prop = ... "for all" ... ;
}
```

One abstract syntax, n concrete syntaxes $\Rightarrow$ n^2 translations, all going through the meaning.

The RGL, and functors

The Resource Grammar Library gives you, per language:

- a **Syntax API** — `mkCN`, `mkNP`, `mkS` — same names everywhere
- a **Paradigms API** — `mkN`, `mkA`, `mkV` — language-specific

So you write the linearization *once*, against interfaces:

```
incomplete concrete MathCoreFunctor of MathCore = open Syntax, Utilities in { ... }

concrete MathCoreFin of MathCore = MathCoreFunctor with
  (Syntax = SyntaxFin), (Utilities = UtilitiesFin) ;
```

"Functors make it maximally easy to port GF grammars into new languages." — *Informath under the Hood*

What a new language needs

`UtilitiesFin`	function words, *instance* of the `Utilities` interface
`CategoriesFin`, `MathCoreFin`, ...	functor instantiations (one-liners)
`VerbalConstantsFin`	∼120 mathematical words
`WikidataWordsFin`	∼2000 mathematical words
`InformathFin`	glue

Three one-liners and two dictionaries.

For Finnish, this is actually true.

Finnish: the RGL is not the problem

The Finnish RGL is mature

	Finnish	Czech
`Paradigms` constructors	55	7
`Paradigms` lines	1090	138
`Sentence` lines	74	32
`Idiom` lines	96	5
`Question` lines	99	7
`Structural` lines	334	31
`Missing<Lang>` stubs	no such module	97 `notYet`

Finnish has `mkV`, `mkVS`, `mkV3`, `mkPN`, `mkSubj`, `mkPredet`, ...

Czech has `mkN`, `mkA`, `mkA2`, `mkV2`, `mkAdv`, `mkPrep`, `mkConj`. No `mkV` at all.

"Language X is in the RGL" is not a binary fact.

And it shows

Finnish morphology, for free, from a one-word paradigm:

```
lin ring_Noun = mkNoun "rengas" ;
```

```
CoreAllProp (NounKind ring_Noun) (StrIdent "R") ...
```

$\Longrightarrow$ *kaikille renkaille R*, ...

Allative plural, with consonant gradation *ng* $\rightarrow$ *nk* and the *-as* stem. Nobody wrote that down.

Compare Czech, where the one-argument `mkN` sends everything in `-ce` — *matice*, *funkce*, *kružnice* — to the *soudce* 'judge' paradigm, and declines them as animate masculine.

So the port should be easy

It was.

And that is the whole problem.

When the compiler stops testing you

Day one: it compiled

The Finnish modules were created as “compilable templates”, by copying.

```
$ gf InformathFin.gf
linking ... OK
```

What was actually in them:

```
-- UtilitiesFin.gf                    -- Swedish
then_Adv       = mkAdv "så" ;
such_that_Subj = mkSubj "så att" ;
arbitrary_A    = mkA "godtycklig" ;
iff_Subj       = mkSubj "om och endast om" ;
proof_Str      = "bevis" ;

-- VerbalConstantsFin.gf              -- English
lin natural_Noun = mkNoun "natural" "number" ;
lin set_Fam      = mkFam "set" ;
lin union_FunC   = mkFun "union" ;
```

The central asymmetry

Czech	**Finnish**
RGL half-built	RGL mature
compiler screams	compiler silent
∼20 crash-driven iterations	`linking ...` `OK` from the start
errors are *structural*	errors are *semantic*
"does it build?"	"is it Finnish?"

A complete RGL removes your only automatic test.

In Czech, every mistake I made stopped the build. In Finnish, a grammar that emits fluent Swedish is indistinguishable, to the compiler, from a correct one.

Three layers of lexicon

Layer 1: `UtilitiesFin` — the function words

∼23 strings. Swedish → Finnish, using the English oper identifiers as the only clue to meaning.

identifier	was (sv)	now (fi)
`then_Adv`	så	niin
`such_that_Subj`	så att	siten että
`arbitrary_A`	godtycklig	mielivaltainen
`iff_Subj`	om och endast om	jos ja vain jos
`proof_Str`	bevis	todistus
`theorem_Str`	teorem	lause
`definition_Str`	definition	määritelmä
`infinity_NP`	äärettön *(a typo, in Swedish's place)*	ääretön

Some strings were *already* Finnish (määritellä, olettaa, tyyppi, funktio, joukko). The file was a sediment of two languages, and compiled either way.

Layer 2: `VerbalConstantsFin` — the maths words

∼120 strings, an English copy with two Finnish words in it.

- **Types**: propositio, numero, totuusarvo, luonnollinen luku, rationaaliluku, reaaliluku, joukko
- **Arithmetic**: summa, erotus, tulo, osamäärä, *korotus . . . potenssiin*, vastaluku, *logaritmi . . . kannassa*, neliöjuuri, itseisarvo, kertoma, suurin yhteinen tekijä
- **Sets**: yhdiste, leikkaus, komplementti, karteesinen tulo, potenssijoukko, osajoukko / aito osajoukko, ylijoukko, alkio, tyhjä joukko
- **Analysis**: pinta-ala, säde, kohtisuora, pituus, normi, mahtavuus, aste, sini / kosini / tangentti, pistetulo, kulma

Note *luonnollinen luku*: the template had `mkNoun "natural" "number"` — two arguments, adjective plus noun. The structure was right. Only the language was wrong.

Layer 3: `WikidataWordsFin` — and the one thing that *did* fail

2051 lins, byte-identical to the English module.

It was the only file that did not compile — and so it had simply been **commented out** of `InformathFin`:

```
concrete InformathFin of Informath =
  MathCoreFin,
  VerbalConstantsFin,
----  WikidataWordsFin,        -- doesn't compile
  ProperNamesFin ;
```

Diagnosis. It referenced the *English morphodict*:

```
lin 'knot_Noun'  = mkNoun knot_N ;        -- knot_N lives in ExtractEng
lin 'abelian_Adj' = mkAdj Abelian_A ;     -- MathWordsFin is a different, smaller set
```

A grammar that "worked" by excluding 2000 of its 2100 words.

The fix: drop the morphodict

Finnish has good smart paradigms. So do not reference lemmas at all — inline the strings:

```
concrete WikidataWordsFin of WikidataWords = CategoriesFin **
  open UtilitiesFin in {          -- no ExtractFin

lin 'knot_Noun'        = mkNoun "solmu" ;
lin 'semigroup_Noun'   = mkNoun "puoliryhmä" ;
lin 'quasicoherent_Adj' = mkAdj "kvasikoherentti" ;
lin 'close_Verb'       = mkVerb "sulkea" ;
```

- `ParadigmsFin.mkN` handles gradation, vowel harmony, stem types.
- The module now needs no morphodict, and compiles standalone.
- Enabled in `InformathFin`. `linking ...  OK` — this time meaning it.

This is exactly the move that is unsafe in Czech, where the one-argument `mkN` guesses wrong for ∼19% of nouns.

Translating 2036 terms with an LLM

Pipeline

1. extract identifiers, split by category
   1216 nouns, 749 adjectives, 86 verbs
2. 7 batches, 7 subagents in parallel
3. each returns `english\tfinnish` TSV
4. merge, check coverage
5. code-generate the module

Result

- full coverage, 0 missing
- 0 malformed, 0 duplicate keys
- every abstract identifier preserved

Spot-checks: ryhmä, rengas, kunta, avaruus, monisto, lyhde (sheaf), ydin (kernel), ominaisarvo, kohomologia, jaoton (prime).

Nominative singular for nouns, A-infinitive for verbs.
One column. Finnish morphology does the rest.

It speaks Finnish

```
VarHypo (StrIdent "x") (NounKind group_Noun)
  ==>  olkoon $ x $ ryhmä

CoreAllProp (NounKind ring_Noun) (StrIdent "R") FalseProp
  ==>  kaikille renkaille $ R $ , meillä on ristiriita

CoreExistProp (NounKind matrix_Noun) (StrIdent "M") FalseProp
  ==>  on olemassa matriisi $ M $ , siten että meillä on ristiriita

CoreIfProp FalseProp FalseProp
  ==>  jos meillä on ristiriita , niin meillä on ristiriita

PropHypo FalseProp
  ==>  oleta , että meillä on ristiriita
```

olkoon — a real jussive, from `ImpP3` in `IdiomFin`.

In Czech, `IdiomCze` was empty and `ImpP3` had to be written by hand.

What the compiler cannot see

Wrongness that links cleanly, I: prepositions

Finnish expresses these relations with *cases*, not prepositions.

```
applied_to_Prep : Prep = mkPrep "sovellettuna" ;
defined_as_Prep : Prep = mkPrep "määriteltynä" ;
as_Prep         : Prep = mkPrep "muodossa" ;
at_Prep         : Prep = mkPrep "kohdassa" ;
over_Prep       : Prep = mkPrep "yli" ;
```

These are pre-positioned words that do not govern the case of their argument.

They **compile**. They **read acceptably**. They are **not right**.

No test in the repository distinguishes this from correct Finnish.

Wrongness that links cleanly, II: the lexicon

- **Coinages.** ∼1500 terms have no established Finnish word.

  `quiver` ⟶ `nuolikko`

  Plausible. Invented. Unreviewed.

- **Capitalisation.** 8 proper-name adjectives kept a capital:

```
lin Abelian_Adj = mkAdj "Abelin" ;    lin Baire_Adj = mkAdj "Bairen" ;
lin abelian_Adj = mkAdj "abelin" ;    -- the same word, twice, two ways
```

- **Irregular stems.** Inline strings rely on smart paradigms. A few will mis-inflect. Nothing reports which.

Every one of these is a silent defect in a grammar that links.

What copying hid

`WikidataWordsFin` was derived from `WikidataWordsEng`. So it inherited English's bookkeeping:

	lins	unique	not in abstract	duplicated
`WikidataWordsEng`	2051	2036	33	15
`WikidataWordsFin`	2051	2036	33	15
`WikidataWordsCze`	2003	2003	0	0

The 33 orphans are exactly the symbol tokens:

```
lin A_Noun = mkNoun "A" ;   lin C222_Noun = mkNoun "C222" ;   lin csgn_Noun = mkNoun "csgn" ;
```

Seven subagents dutifully translated identifiers that the abstract syntax does not contain. GF only *warns*. The Czech module was generated *from the abstract* — so it cannot drift.

Lessons

What the LLM was good at

- **Bulk lexicography.** 2036 terms, 7 agents, minutes. The mathematical vocabulary is genuinely good: *lyhde* for sheaf, *ydin* for kernel, *jaoton* for prime.
- **Diagnosis.** Working out that `WikidataWordsFin` failed because `knot_N` lives in `ExtractEng` and `MathWordsFin` is a different, smaller extraction — then proposing to drop the morphodict entirely.
- **Noticing the sediment.** That `UtilitiesFin` was Swedish-with-Finnish-patches, and that `äärettön` was a typo for `ääretön`.
- **Mechanical faithfulness.** Every abstract identifier preserved exactly, through a code-generated module.

What it was bad at

- **Treating `linking ... OK` as success.**
  It is, at best, evidence that the file is well-formed GF.
- **Volunteering that `mkPrep "yli"` is not how Finnish works.**
  It compiled, so it was reported as done. The caveat exists because somebody asked afterwards.
- **Questioning its input.** It translated all 2051 lins, including 33 that no `fun` references and 15 that were already there. It was given a file and it translated the file.
- **Knowing which coinages are real.** *nuolikko* and *lyhde* are presented with identical confidence. One is standard.

Lessons for the next language

1. **Check the RGL first — but for the opposite reason.**
   A *poor* RGL costs you weeks of resource-grammar work. A *good* RGL costs you your safety net.
2. **Generate from the abstract syntax, never copy a sibling.**
   Copying gave Finnish 33 dead lins and 15 duplicates, for free, silently. Code-generating Czech from `WikidataWords.gf` made "complete" a checkable property.
3. **Build the test the compiler will not give you.**
   Coverage, category, morphology class. Reject, do not repair.
4. **Smart paradigms are a property of the RGL, not of the LLM.**
   Trust them in Finnish. Do not trust them in Czech.
5. **Separate "the model produced it" from "it is right".**
   The 2036 Finnish terms have had no native review. That sentence belongs in the commit message.

Still open

- **Native review of ∼2036 translations**, especially the rare coinages.
- Convert the `mkPrep` string relations to case-governed prepositions.
- Lowercase the proper-name adjectives (*Abelin* / *abelin*), and decide which of the two duplicates survives.
- Add explicit paradigms where the smart guess mis-inflects an irregular stem.
- Purge the 33 orphan lins — in English as well as Finnish.

Conclusion

Czech was hard because the compiler kept stopping me.

Finnish was hard because it never did.

A mature RGL delivers exactly what it promises: the functor instantiates, the morphology is free, and a two-thousand-word lexicon is an afternoon.

What it also delivers is a grammar that compiles, links, and speaks Swedish — and no way to tell, except by reading it.

`github.com/GrammaticalFramework/informath`